Software Engineering

UML Sequenzdiagramm

Prof. Dr.-Ing. Sven Graupner



UML Sequenzdiagramm

1. Lebenslinie
2. Ereignis und Ausführungen
3. Nachrichten
   - Synchron (3.1), Asynchron (3.2), Multicast (3.3)
4. Verarbeitung von Nachrichten
   - Sequentiell (4.1), Nebenläufig (4.2)
5. Antwort-Nachrichten
   - Asynchrone Antwort (5.1), Asynchroner (5.2), Synchroner (5.3) Callback
6. Zusammenfassung

UML Sequenzdiagramm

Das **Sequenzdiagramm** beschreibt den logischen zeitlichen Ablauf des Nachrichtenaustausch zwischen Beteiligten/Komponenten.

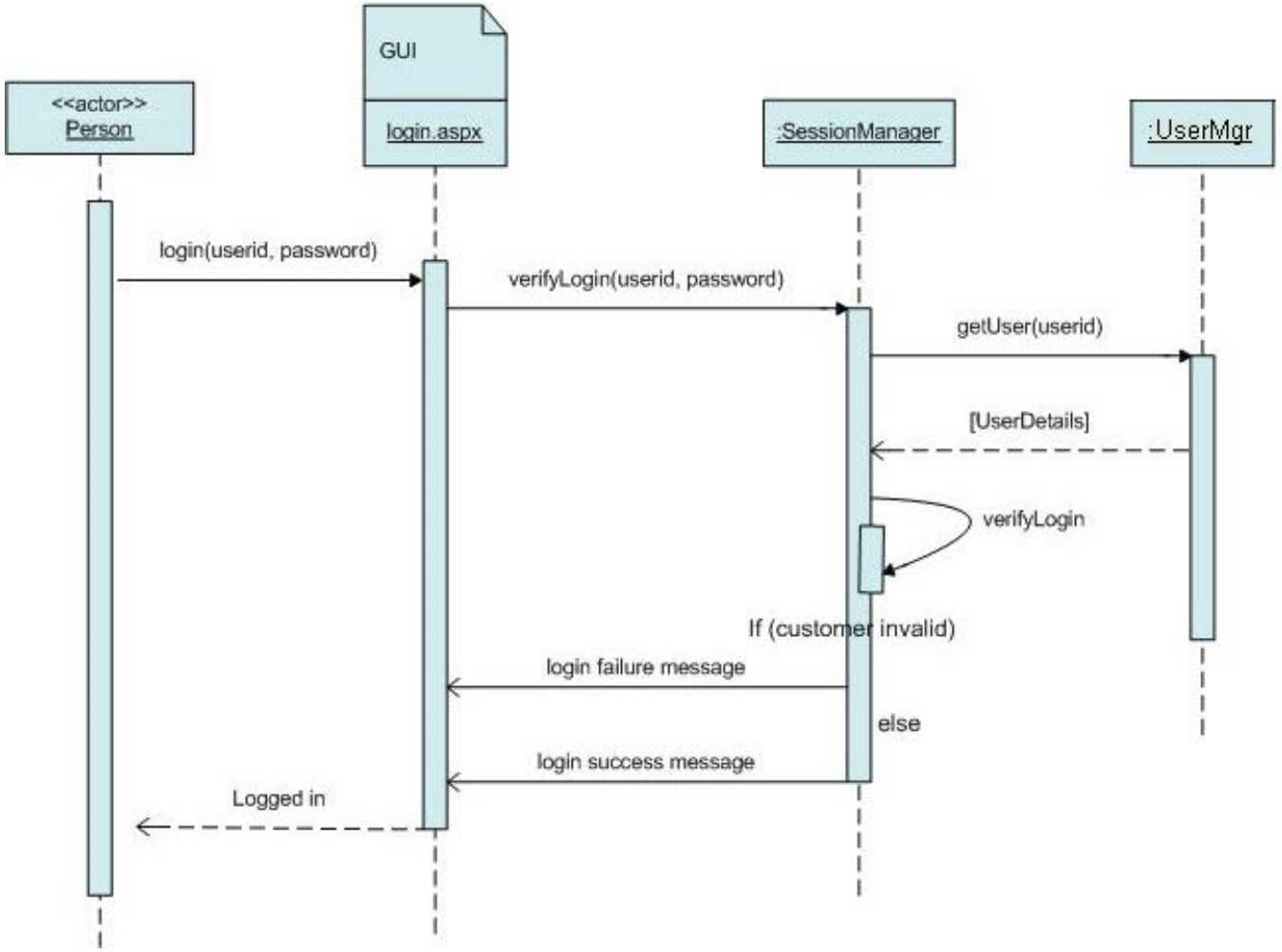
Logischer zeitlicher Ablauf bedeutet korrekte Darstellung der Reihenfolge ohne Angabe von Zeiträumen (→ Zeitverlaufsdiagramm).

Der zeitliche Ablauf wird im Sequenzdiagramm über Lebenslinien von (life lines) dargestellt.

UML Sequenzdiagramm

Beispiel:

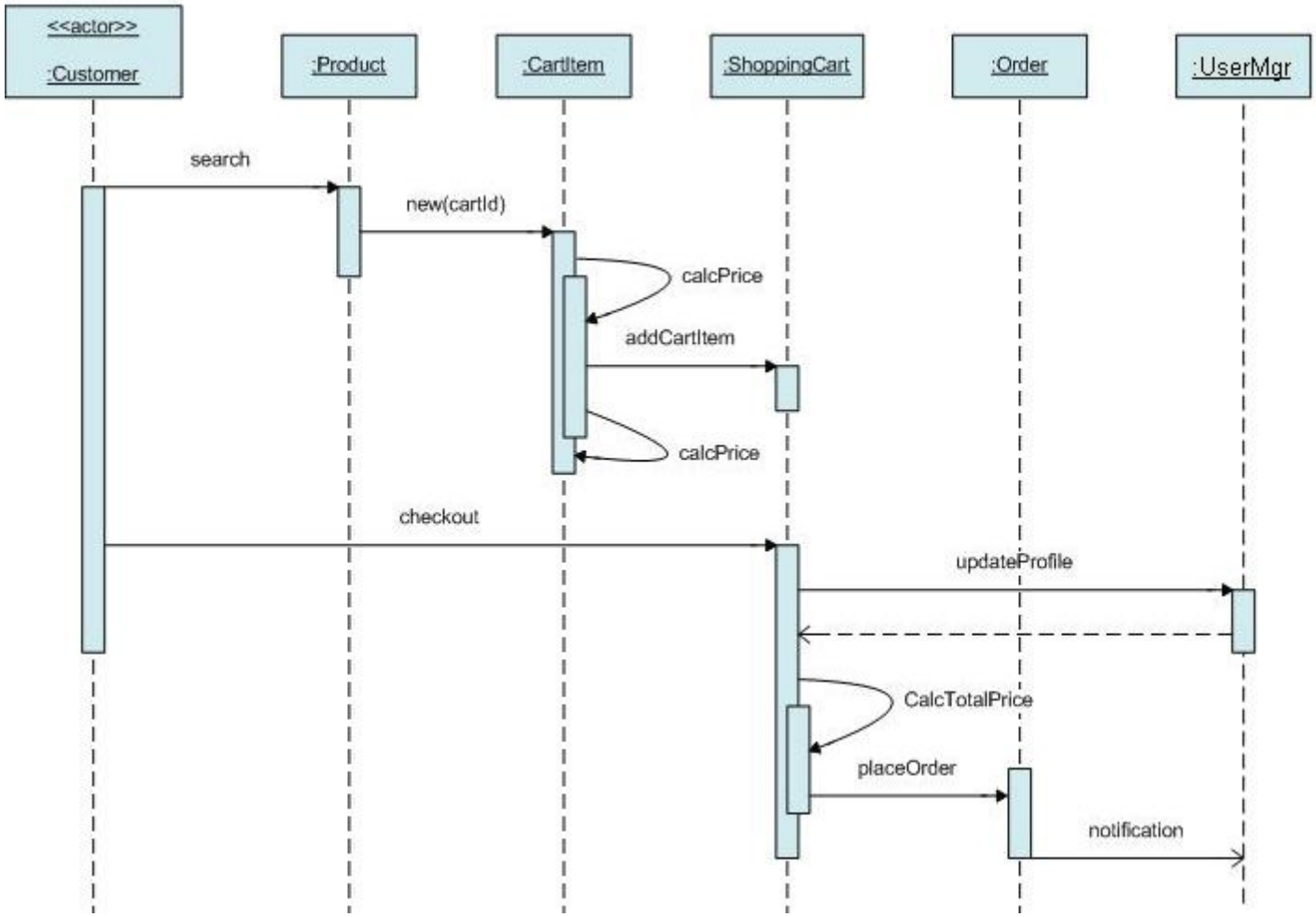
User-
Login



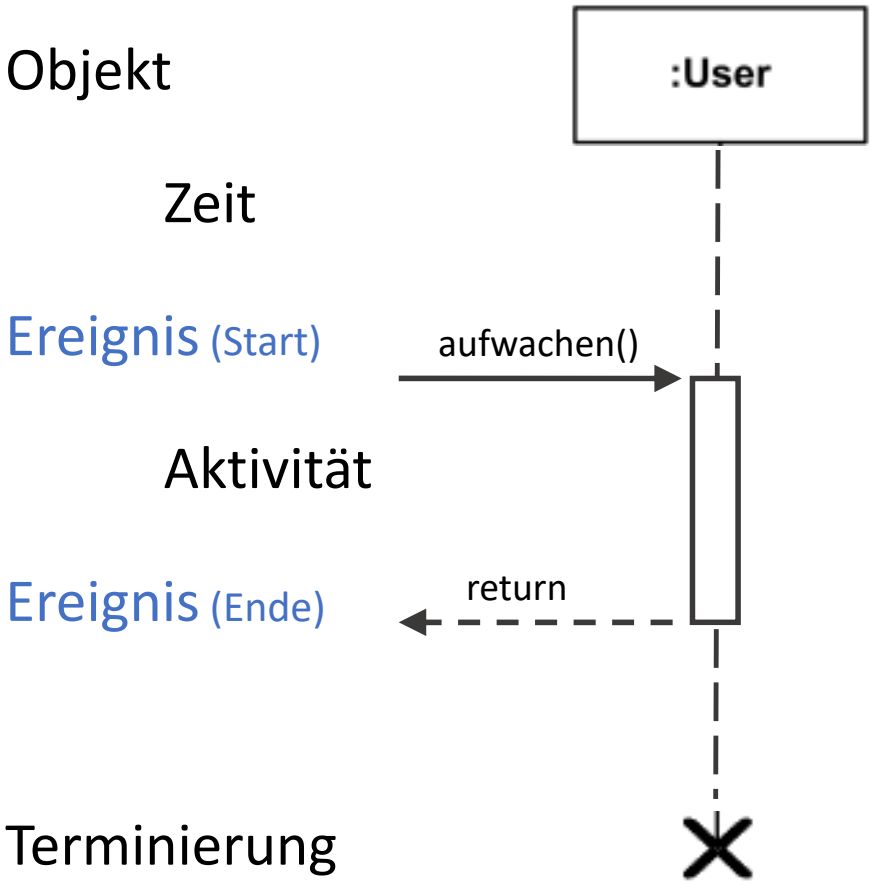
UML Sequenzdiagramm

Beispiel:

Shopping



1 Lebenslinie

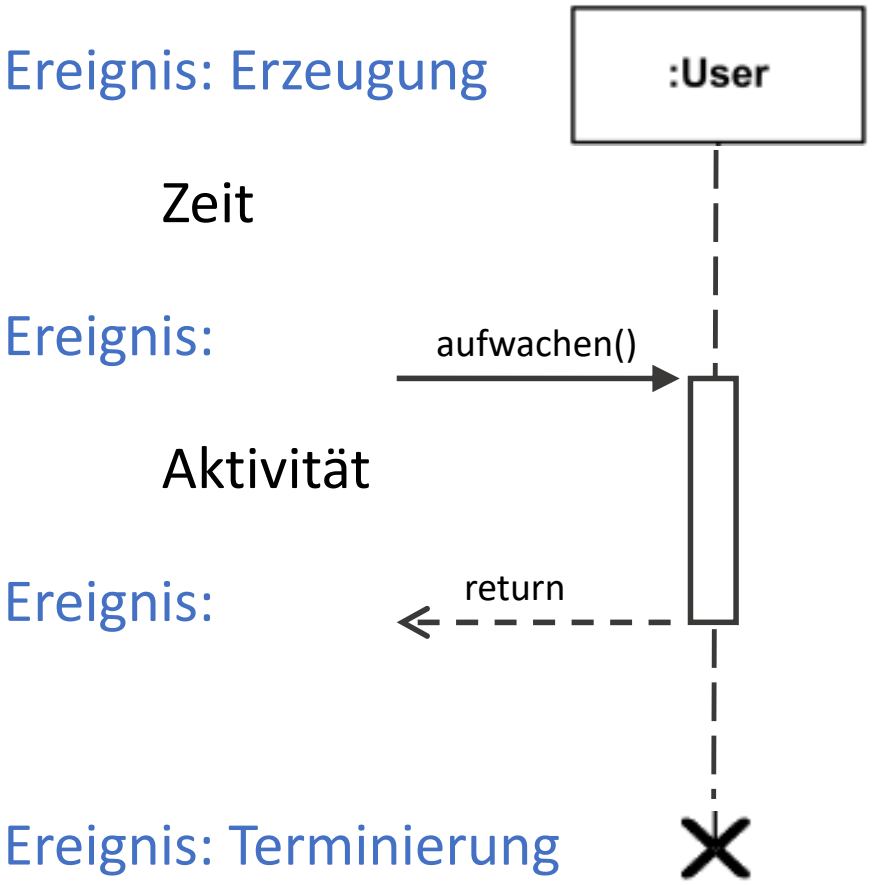


Eine Lebenslinie ist stets mit einem Objekt (oder Objekten einer Klasse) verbunden.

Sie beginnt mit der Erzeugung des Objektes und endet mit seiner Existenz.

Dazwischen werden Phasen von Passivität und Aktivität unterschieden, die durch Ereignisse (“occurences”) bedingt werden.

1.1 Lebenslinie mit Ereignissen



Eine Lebenslinie ist stets mit einem Objekt (oder Objekten einer Klasse) verbunden.

Sie beginnt mit der Erzeugung des Objektes und endet mit seiner Existenz.

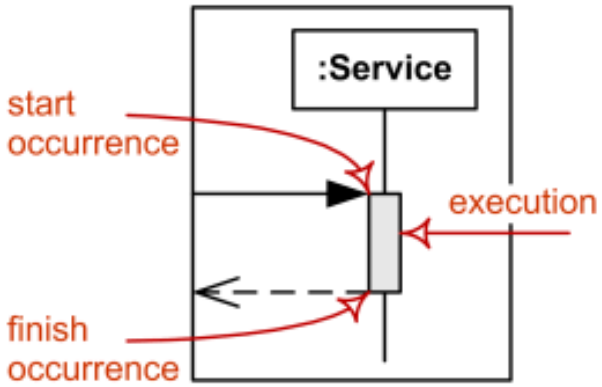
Dazwischen werden Phasen von Passivität und Aktivität unterschieden, die durch Ereignisse (“occurences”) bedingt werden.

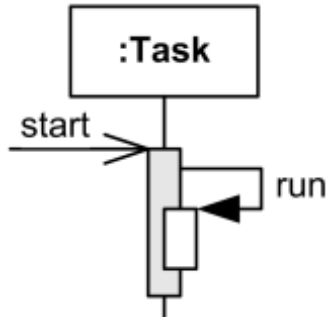
2 Ereignisse und Ausführungen

Ereignisse (occurrences) repräsentieren Zeitpunkte an denen Aktionen (actions) beginnen oder enden.

Ausführungen (executions) bedeuten eine Phase der Aktivität, die mit Ereignissen (execution occurences) beginnt oder endet.

Eine Ausführung kann “sich selbst aufrufen”.





3 Nachrichten

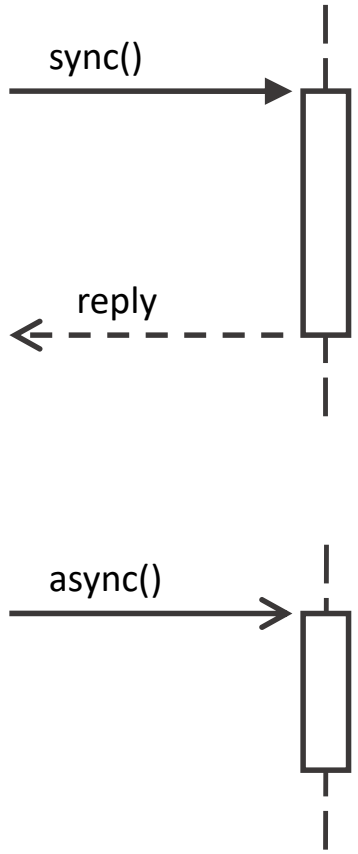
Es werden

- **synchrone** und
- **asynchrone**

Nachrichten unterschieden. Nachrichten können Aufrufe (Requests) sein, denen

- **Antwort-Nachrichten** (Return, Reply)

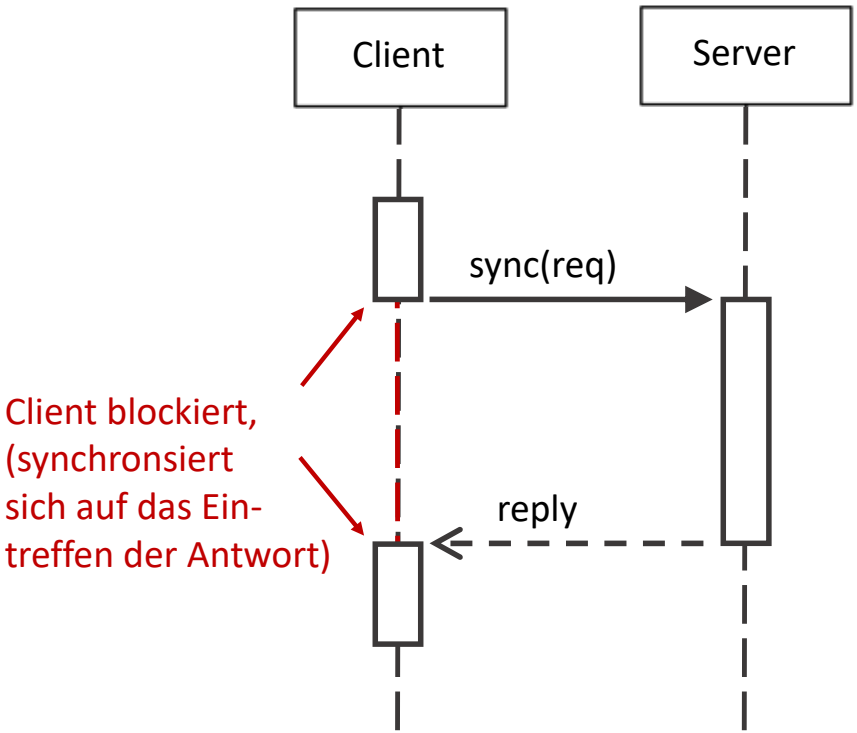
folgen.



3.1 Synchrone Nachricht

Synchrone Nachricht

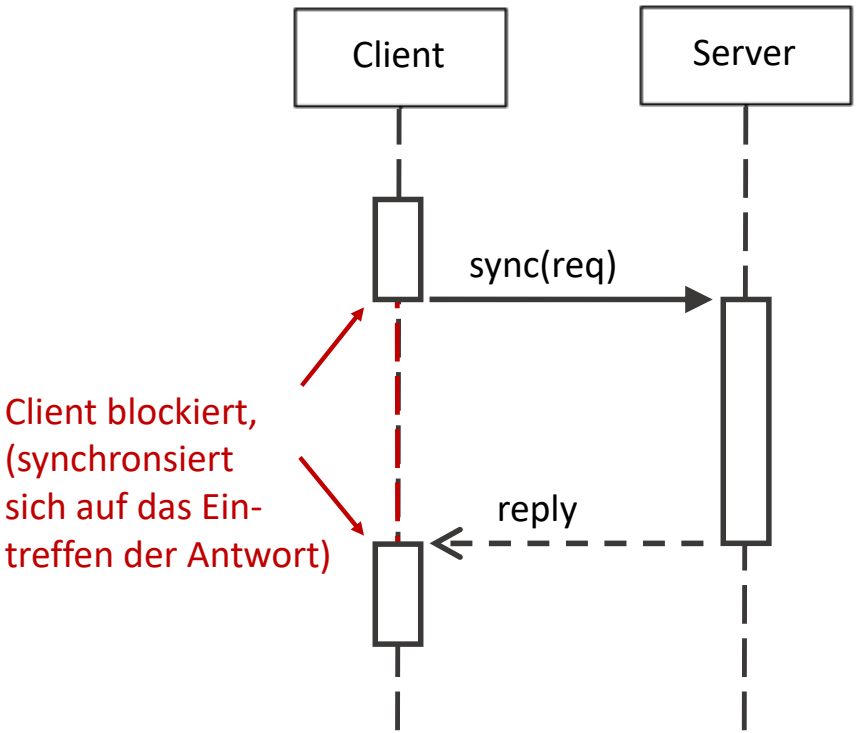
Remote Procedure Call (RPC)



3.2 Asynchrone Nachricht

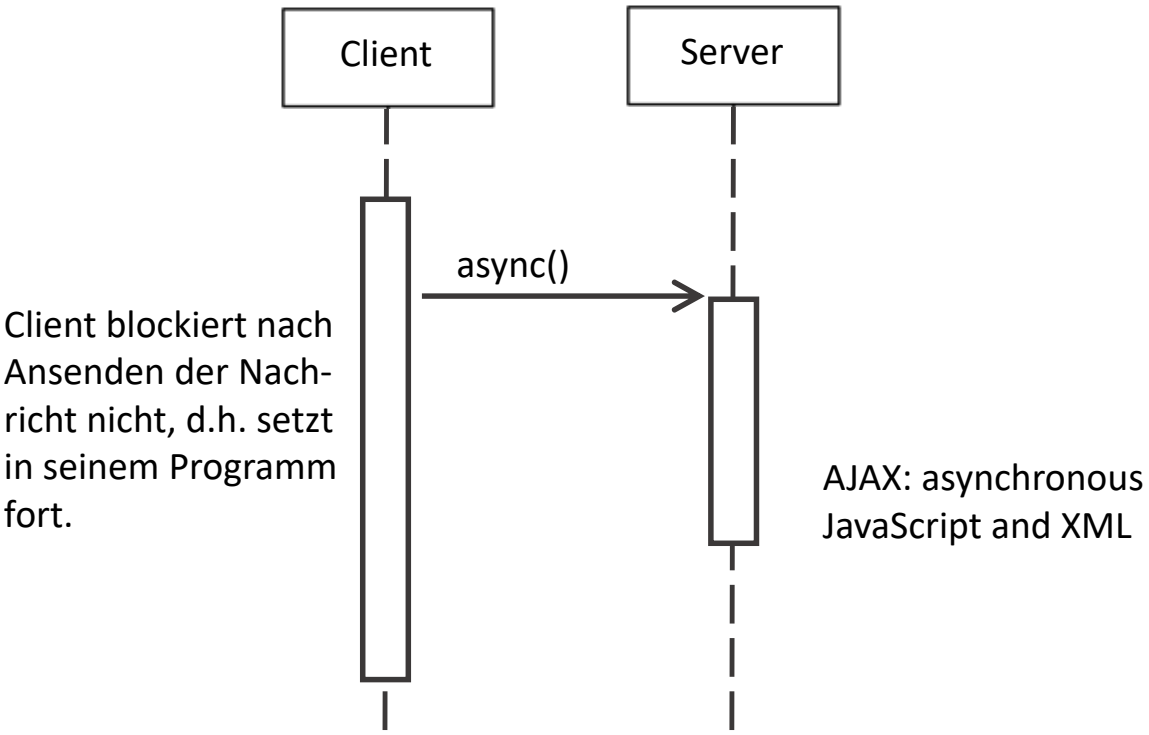
Synchrone Nachricht

Remote Procedure Call (RPC)



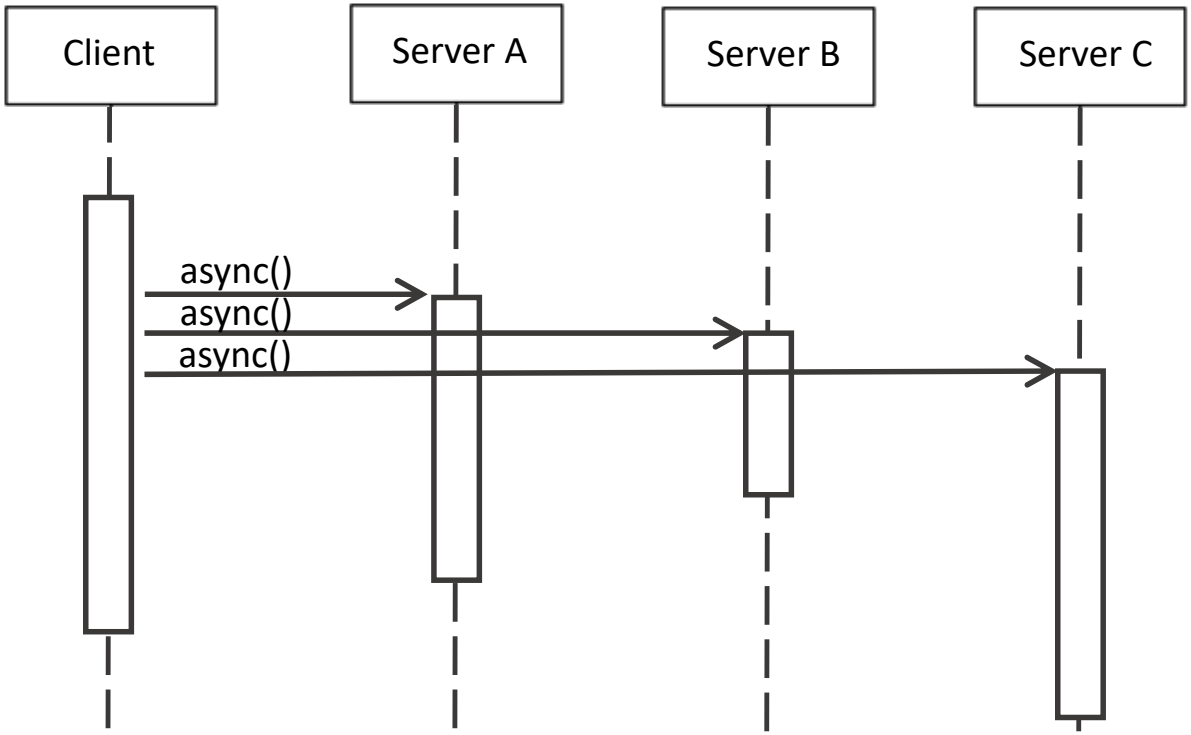
Asynchrone Nachricht

(unsynchronisierter Absender)



3.3 Multicast

Anstossen mehrerer asynchroner Aktivitäten (alle Server bekannt).



4 Verarbeitung von Nachrichten

Nachrichten müssen bei ihrem Eintreffen verarbeitet werden.

Man unterscheidet die

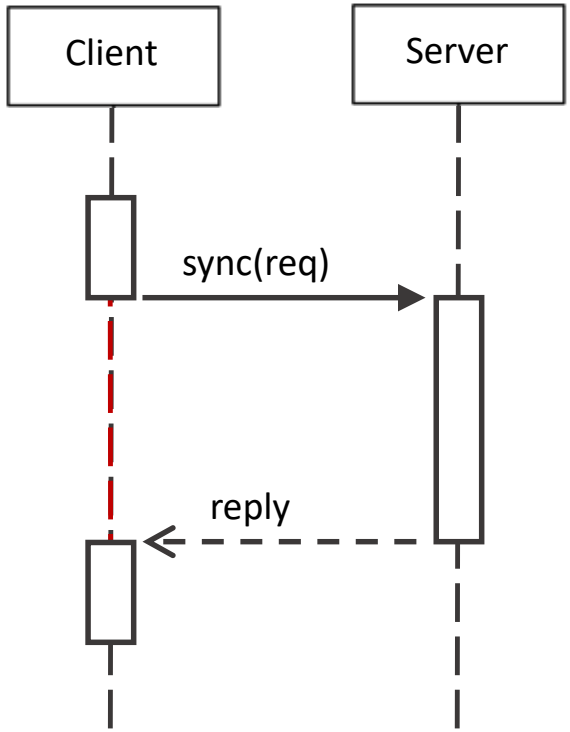
- sequentielle und die
- Nebenläufige Verarbeitung.

Bei der sequentiellen Verarbeitung kann

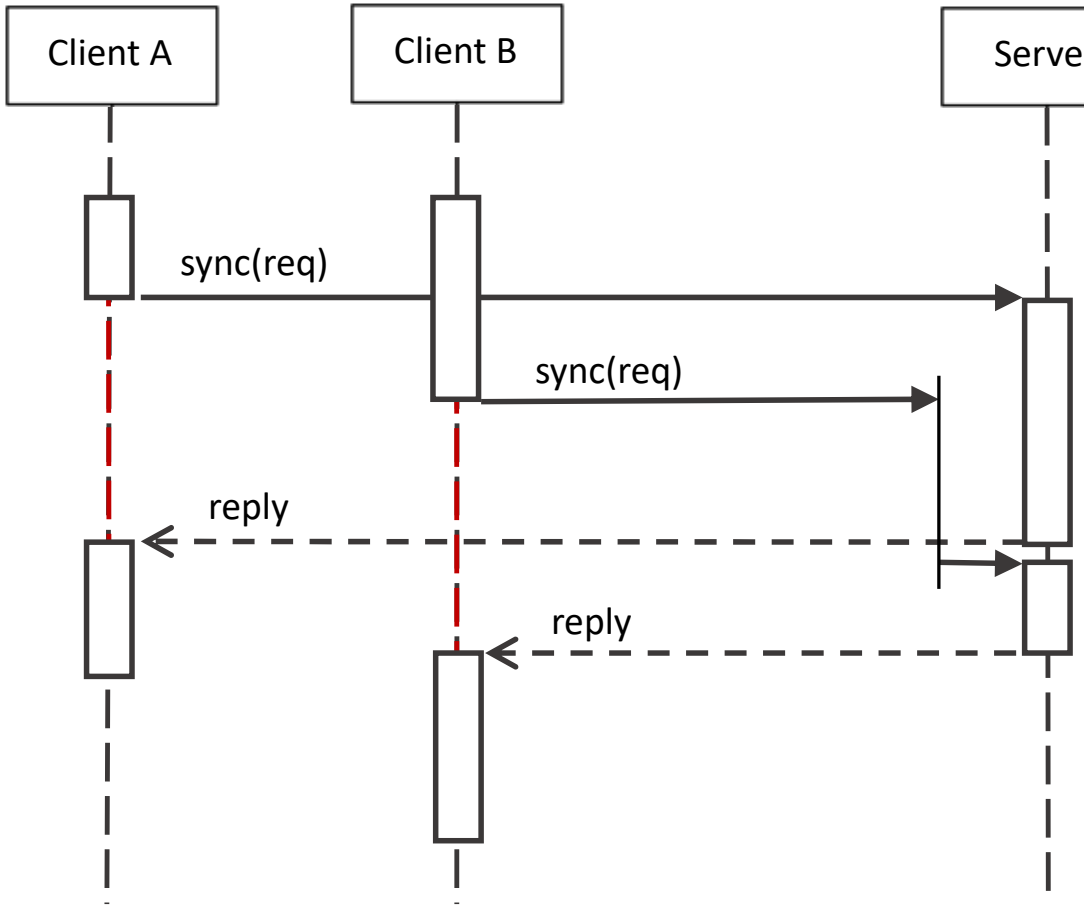
keine weitere Nachricht verarbeitet werden,

wenn sich eine andere Nachricht noch in

Verarbeitung befindet.



4.1 Sequentielle Verarbeitung von Nachrichten

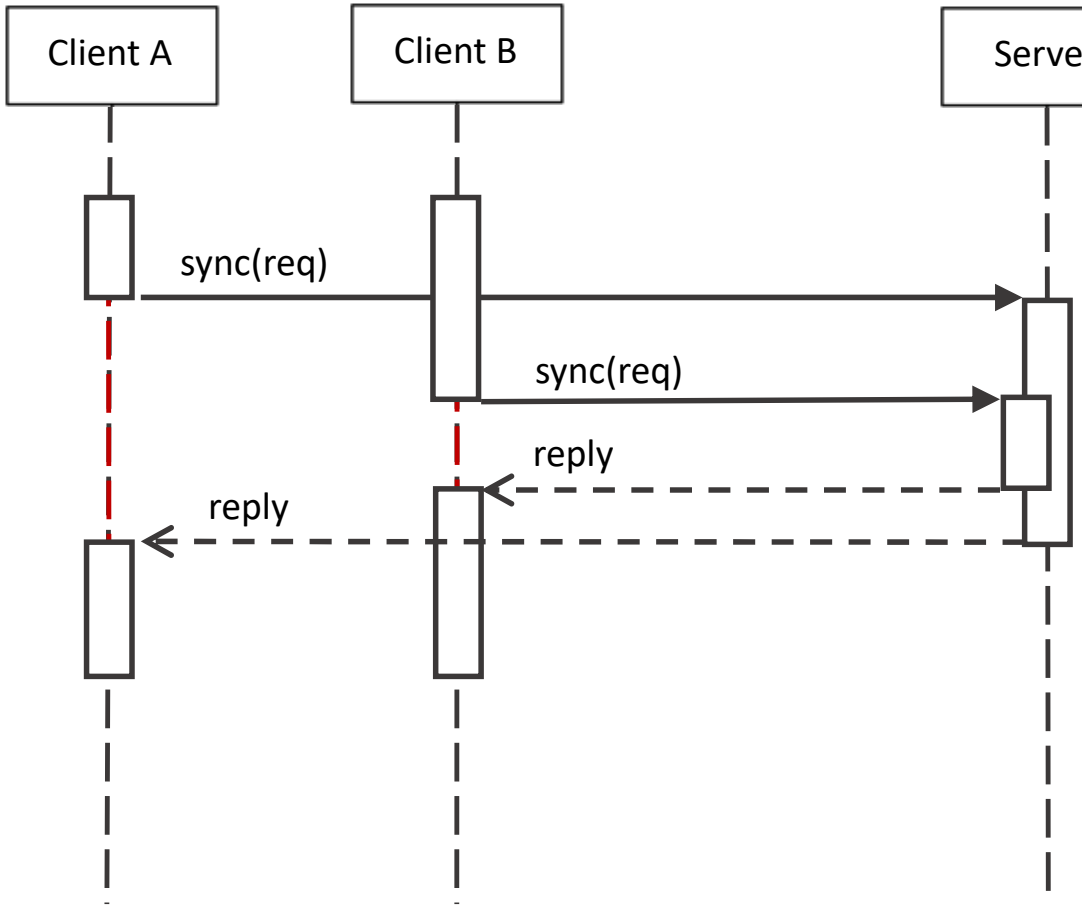


Sequentieller Server.

Kann nur einen Aufruf zu einem Zeitpunkt verarbeiten.

Führt zur Verzögerung der Bearbeitung des Aufrufs von Client B und zu längerer Wartezeit von Client B.

4.2 Nebenläufige Verarbeitung von Nachrichten



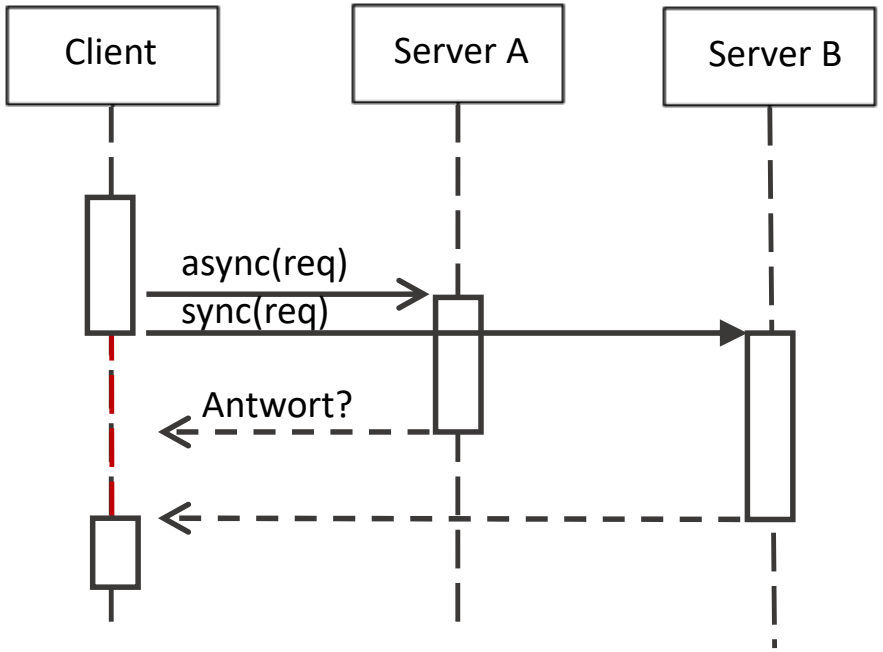
Nebenläufiger Server.

Kann mehrere Aufrufe gleichzeitig verarbeiten (z.B. multi-threaded).

Client B erhält damit im Bsp. seine Antwort sogar früher als Client A.

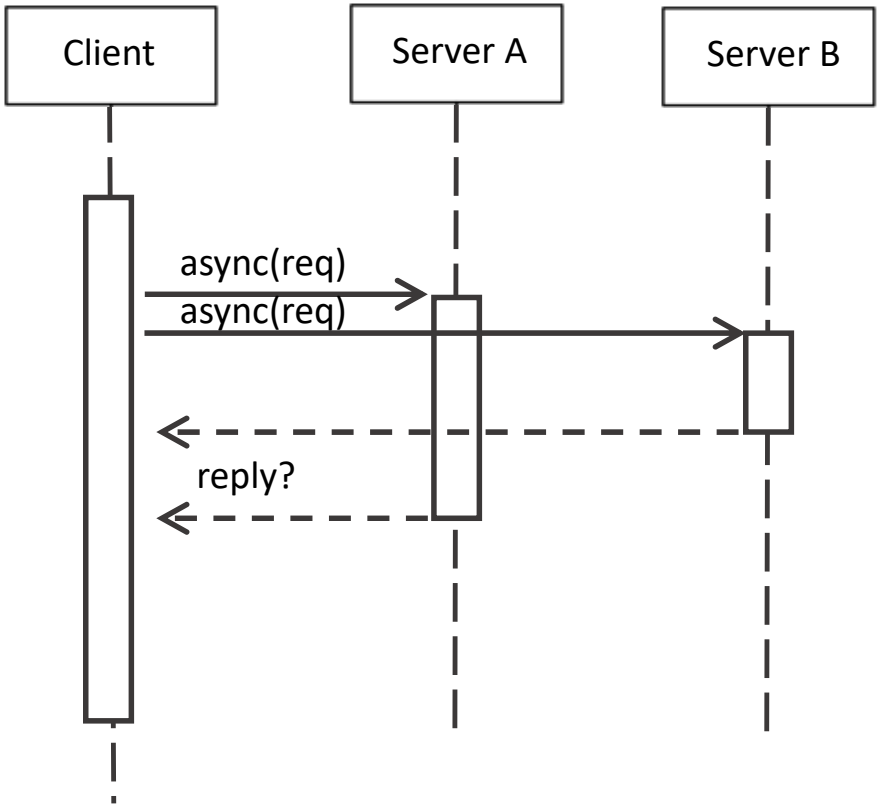
5 Antwort-Nachrichten

Bislang gab es keine Rückantworten bei asynchronen Aufrufen und der Client blockierte, bis ein Antwort auf einen synchronen Aufruf eintraf.



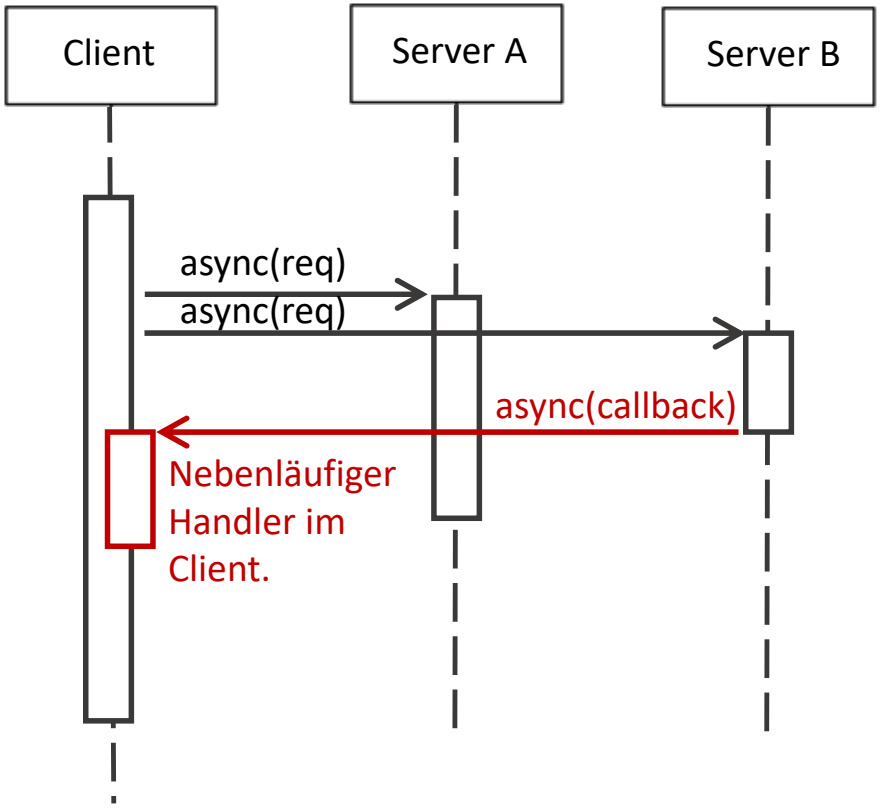
5.1 Asynchrone Antwort

Bislang gab es keine Rückantworten bei asynchronen Aufrufen.



5.2 Asynchrone Antwort als Callback

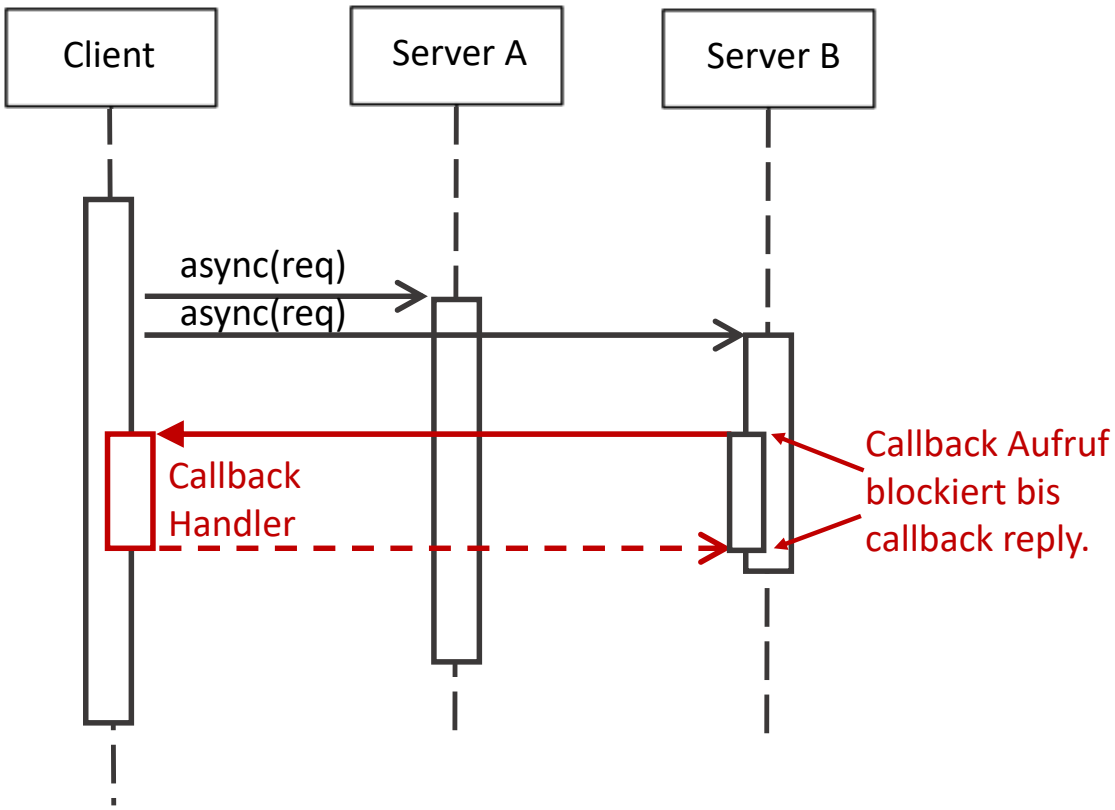
Lösung: nebenläufiger Client.



Der Client muss jederzeit in der Lage sein, eintreffende Antworten als sync oder async Nachrichten zu verarbeiten.

Dazu muss er, wie ein nebenläufiger Server, Code gleichzeitig (z.B. in einem neuen Thread) ausführen.

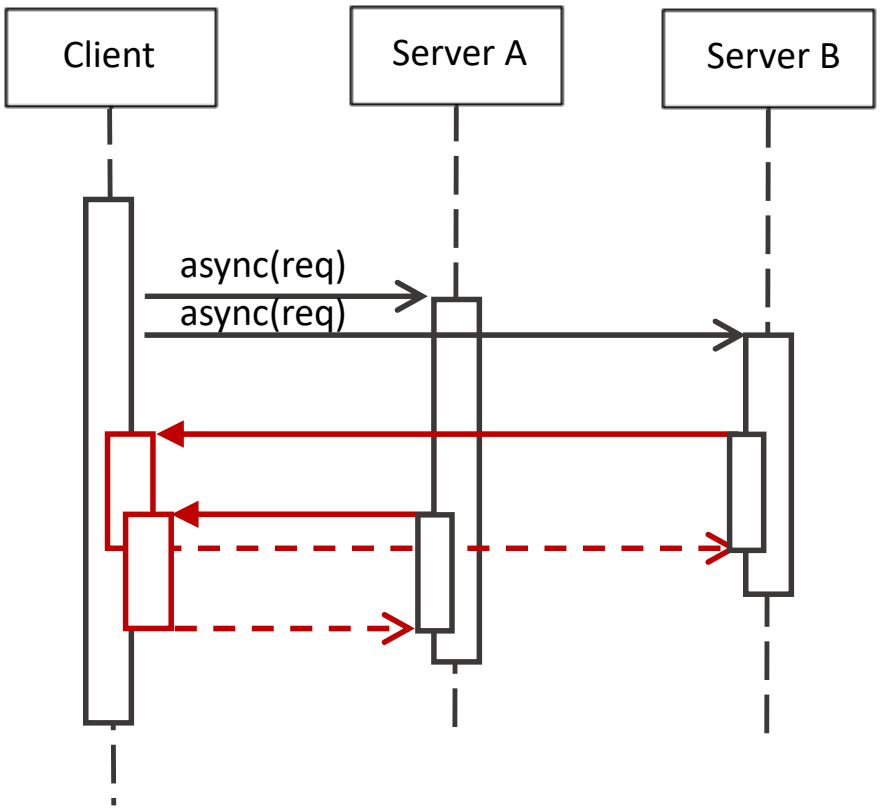
5.3 Synchrone Antwort als Callback



Der Client muss jederzeit in der Lage sein, eintreffende Antwort-nachrichten (replies, sync + async) zu verarbeiten.

Dazu muss er, bei einem parallelen Server, Code nebenläufig z.B. in einem neuen Thread ausführen.

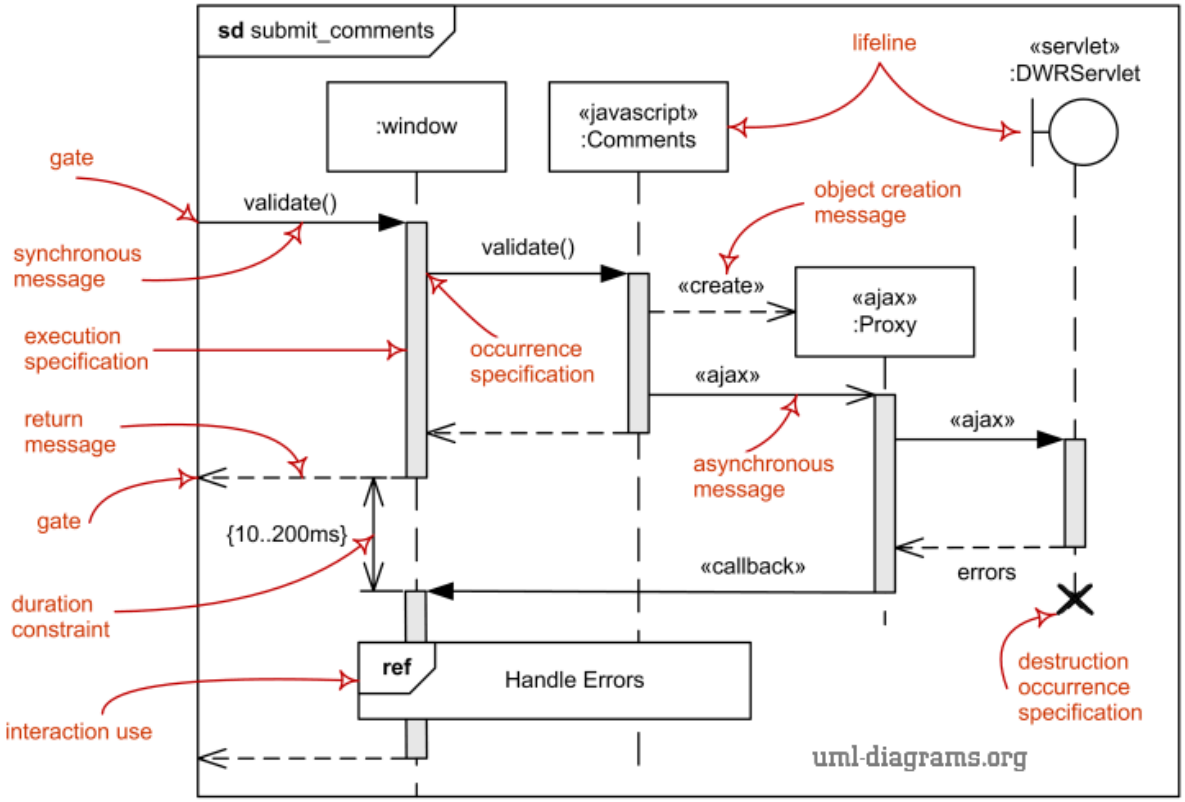
5.3.1 Synchrone Antwort als Callback



Der Client muss jederzeit in der Lage sein, eintreffende Antwort-nachrichten (replies, sync + async) zu verarbeiten.

Dazu muss er, bei einem parallelen Server, Code nebenläufig z.B. in einem neuen Thread ausführen.

Zusammenfassung: UML Sequenzdiagramm



Anhang: Weitere UML Interaktionsdiagramme

Neben dem Sequenzdiagramm gibt es weitere UML Interaktions-diagramme:

- Zeitverlaufsdiagramm (Timing Diagram)
- Kommunikationsdiagramm
- Interaktionsübersichtsdiagramm

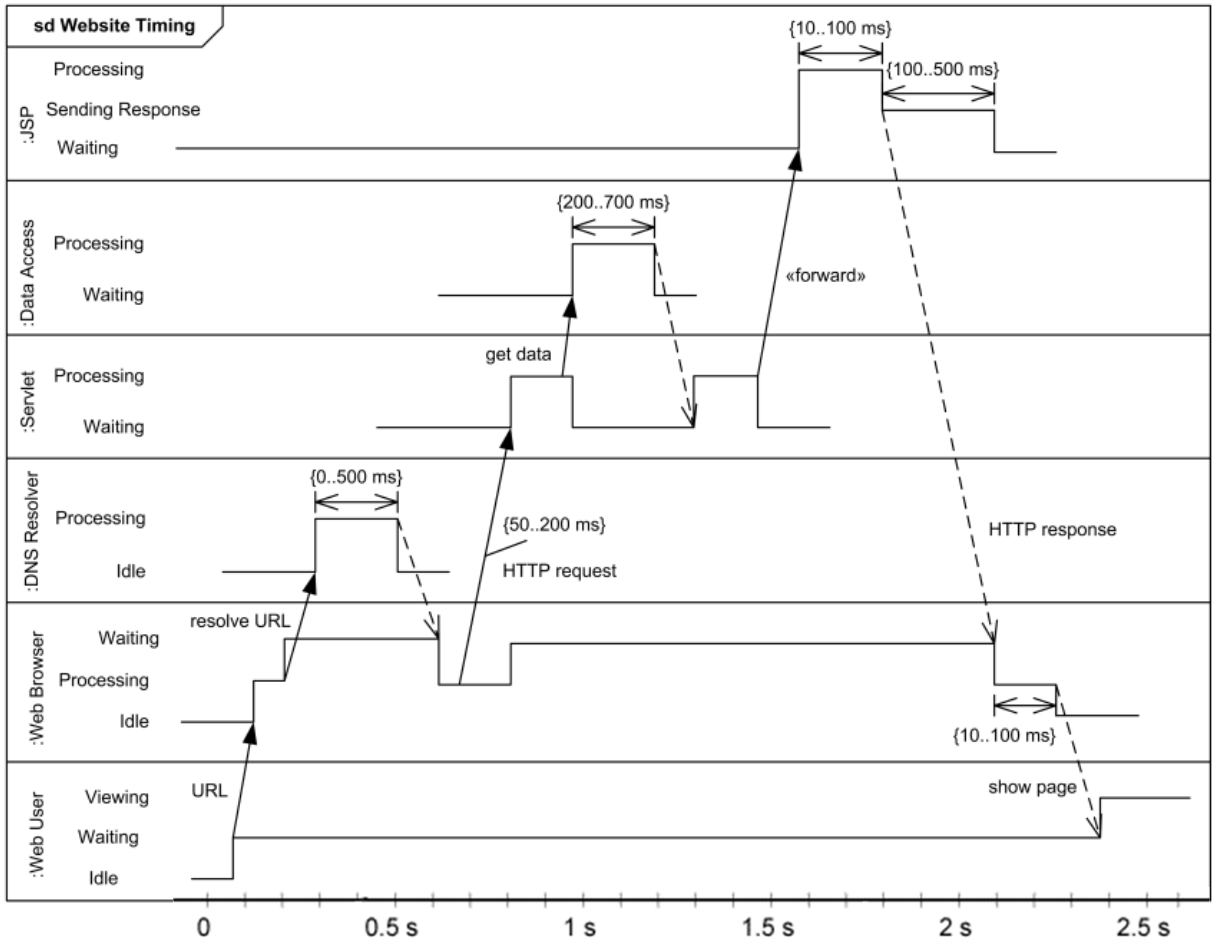
a) UML Zeitverlaufsdiagramm

In einem **Zeitverlaufsdiagramm** wird (im Unterschied zu einem Sequenzdiagramm) nicht nur der logische Ablauf von Ereignissen, sondern ein zeitlicher Maßstab dargestellt.

Zeiträume wie Lebenslinien von Komponenten werden auf eine **gemeinsame Zeitachse** bezogen.

Damit werden zeitliche Relationen sichtbar.

Beispiel: Zeitverlauf eines Web-Requests



- **JSP Java Server Pages** ist eine Komponente in einem Java-Web Server, welche HTML Seiten generiert, die in der HTTP Response zurückgeschickt werden.
- **Data Access** ist ein Layer im Web Server, welcher den Zugriff zu persistenten Speichern z.B. zu Datenbanken realisiert.
- **Servlet** ist eine Komponente in einem Java Web Server, welche HTTP Requests entgegennimmt und deren Verarbeitung im Server initiiert.
- **DNS Resolver** ist ein Server, welcher DNS Namen aus URL in IP Adressen auflöst, um einen HTTP Request via TCP/IP an einen Web Server senden zu können.
- **Web Browser** ermöglicht, HTML Seiten anzusehen und auf die in ihnen enthaltenen Links zu clicken, wodurch ein HTTP-GET request an den in der URL des Links enthaltenen Web Server abgesendet wird.
- **Web User** ist eine Person, die auf einen Link geclickt hat und auf die Antwort des Servers wartet.

Zusammenfassung: UML Zeitverlaufsdiagramm

